编程能力建设 · 2026 秋

从「做不出程序」到「能写工程」

六个项目阶梯 · 每级只加一个新维度 · 脚手架递减 · 每天洛谷 3 题打底

先把问题说准：你的缺口不是语法

「C 语言课过了，但连最简单的程序都做不出来」——这不矛盾，因为它们是两种不同的能力。

- **你练过的**：C 语言课的作业几乎全是「补全这个函数」「输出这个图形」「求这个数列的和」。这类练习**已经告诉你第一步做什么、第二步做什么**，你只需要把语法填进去。
- **你没练过的**：给你一句模糊的需求（「做个能记账的东西」），你要自己想清楚：**数据怎么存？分成几个功能？每个功能对应哪个函数？输入错了怎么办？**——这叫程序构造。
- **打个比方**：你背熟了所有砖块的名称（int、for、if、数组、指针），但从来没有盖过房子。所以一动手就不知道第一块砖放哪，这不是能力问题，是**练习类型缺失**。
- **结论**：纯刷题解决不了这件事。刷题题目把输入输出描述得清清楚楚，不需要你做需求分析。**刷题练「算法实现」，做项目才练「程序构造」**，两个都要，但项目是主线。

三类能力，分开练

计算机专业真正的核心能力有三个方向，混在一起练会互相拖累。

能力	表现	怎么练	什么时候
算法实现	给明确输入输出，写出正确代码	洛谷题单，从【入门1】开始，每天 3 题	贯穿全程
程序构造	给模糊需求，设计出程序结构	**做小项目**，六级阶梯，从 80 行到 250 行	**主线，10 月起**
代码阅读	看懂别人的代码在干什么	读小型开源项目，画流程图，逐行标注	寒假开始

为什么代码阅读被排到最后：读别人的代码需要先有「自己写过」的经验做锚点。你连 200 行的程序都没构造过，去读 cJSON 那种一千行的项目只会更挫败。**先自己盖一间小房子，再去看别人怎么盖楼。**

第 0 步：环境（已完成）

本机原先只有 VS Code、没有编译器——这意味着你从来没有「随手跑代码」的条件。这个摩擦大到足以让人永远不开始。

- **已装**：w64devkit（MinGW-w64 便携版），位置 `C:\w64devkit`，含 gcc / g++ / gdb / make。已加入 PATH。
- **验证**：打开终端输入 `gcc --version`，能看到版本号就说明通了。
- **编译运行（命令行，最快）**：
  `gcc hello.c -o hello.exe` 然后 `.\hello.exe`
  养成加 `-Wall -Wextra` 的习惯：`gcc -Wall -Wextra hello.c -o hello.exe`，它会提前告诉你哪里写错了。
- **VS Code 里一键编译**：装扩展 C/C++（Microsoft 官方），然后在项目文件夹下建 `.vscode\tasks.json`，内容见下面。**之后按 Ctrl+Shift+B 就编译，F5 就调试。**

.vscode\tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build",
      "type": "shell",
      "command": "gcc",
      "args": ["-Wall", "-Wextra", "-g", "${file}", "-o", "${fileDirname}\\${fileBasenameNoExtension}.exe"],
      "group": { "kind": "build", "isDefault": true }
    }
  ]
}
```

关键不是装了什么，是那个循环：改一行 → 编译 → 看结果，全程 5 秒内。这个循环跑顺了，学习速度会快一个数量级；跑不顺，所有学习都是纸上谈兵。

六级项目阶梯

每一级**只加一个新维度**（输入 → 状态 → 持久化 → 数据结构 → 工程结构 → 语言抽象），不会一次淹没你。勾选自动保存。

1 个税计算器

约 80 行 · 新维度：**输入** → **判断** → **输出** · 我给完整代码

要做什么	输入税前月工资，按七级累进税率算出个税和税后收入，输出一张明细表
新学到	`scanf` 的**返回值检查**（这是新手最容易漏的一步）、`double` 与 `%lf`、`if-else` 链、`printf` 宽度与精度控制
验收标准	输入 10000 → 应纳税所得 4000 → 个税 190 → 税后 8810。**四个数都对，才算过**

完整代码（先跑通，再改造）

```
/* 个税计算器 —— 中国工资薪金七级累进税率（简化版） */
#include <stdio.h>

int main(void)
{
    double salary;    /* 税前工资 */
    double social;    /* 三险一金 */
    double taxable;   /* 应纳税所得额 */
    double tax = 0;   /* 个税 */
    double after;     /* 税后到手 */

    printf("请输入税前月工资（元）：");

    /* 关键：scanf 返回"成功读到了几个值"，不等于 1 说明输入不是数字 */
    if (scanf("%lf", &salary) ≠ 1) {
        printf("输入有误，请输入数字。\n");
        return 1;
    }
    if (salary ≤ 0) {
```

```
        printf("工资必须大于 0。\n");
        return 1;
    }

    social  = salary * 0.10;              /* 简化：三险一金按 10% 算 */
    taxable = salary - social - 5000.0;   /* 起征点 5000 */
    if (taxable < 0) taxable = 0;          /* 工资低时不能是负数 */

    /* 七级累进税率：税率 + 速算扣除数 */
    if      (taxable ≤ 3000)  tax = taxable * 0.03;
    else if (taxable ≤ 12000) tax = taxable * 0.10 - 210;
    else if (taxable ≤ 25000) tax = taxable * 0.20 - 1410;
    else if (taxable ≤ 35000) tax = taxable * 0.25 - 2660;
    else if (taxable ≤ 55000) tax = taxable * 0.30 - 4410;
    else if (taxable ≤ 80000) tax = taxable * 0.35 - 7160;
    else                      tax = taxable * 0.45 - 15160;

    after = salary - social - tax;

    printf("\n———————— 计算结果 ————————\n");
    printf("税前工资   : %10.2f 元\n", salary);
    printf("三险一金   : %10.2f 元\n", social);
    printf("应纳税所得 : %10.2f 元\n", taxable);
    printf("应缴个税   : %10.2f 元\n", tax);
    printf("税后到手   : %10.2f 元\n", after);
    printf("————————————————————————\n");

    return 0;
}
```

三个改造任务（按顺序做，做完打勾）
① 把 10% 改成让用户输入（体会「硬编码」和「参数」的区别）
② 加一个循环，让程序能连续算多个人，输入 0 才退出（体会 `while` + 提前终止）
③ 加年终奖：年终奖单独计税（用「除以 12 找税率，再全额乘」的算法）——**这个需要你自己查政策，是第一次真正的「需求分析」**

2 记账本 v1（纯内存）

约 150 行 · 新维度：**状态**（数据要活到下一次操作） · 我给骨架

功能	菜单循环：**1 记一笔 / 2 看总额 / 3 列明细 / 4 按类别汇总 / 0 退出**
新学到	**数组**（数据存在哪）、**菜单循环**（`while` + `switch` 的经典组合）、**计数器**（当前存了几笔）、字符串比较 `strcmp`
验收标准	连续记 5 笔不同类别的账，总额和分类汇总都正确；输错菜单号不崩溃
常见坑	① 数组越界（记满 100 笔还继续记）② `switch` 忘了 `break` ③ 用 `==` 比较字符串（C 语言里必须用 `strcmp`）

这一级的真正难点不是语法，是设计：你要先想清楚「一笔账要存哪些字段」（金额、类别、日期、备注？），再决定「用什么装这些账」（几个平行数组？还是先各存各的？）。**把这个想清楚再动手，比写完再改快十倍。**

3 记账本 v2（存文件）

约 200 行 · 新维度：**持久化**（关掉再打开数据还在） · 我给接口说明

要加什么	启动时从文件读入全部账目，退出时写回文件；文件不存在时不能崩
新学到	`fopen / fprintf / fscanf / fclose`、**文件格式设计**（一行一笔还是 CSV？）、**错误处理**（`fopen` 返回 `NULL` 怎么办）
验收标准	记 3 笔 → 退出 → 重新打开 → 3 笔还在，且总额一致
常见坑	① 忘了 `fclose` 导致数据没写进去 ② 文件路径用了相对路径，换个目录运行就找不到 ③ 读文件时没判断读到结尾（`fscanf` 返回值）

这是跨越性的一步：前面两级的程序「跑一次就没了」，这一级开始你的程序有了**记忆**。从这之后，「程序」和「一段计算过程」才真正分家。

4 通讯录

约 250 行 · 新维度：**数据结构**（struct） · 只给需求

功能	**增 / 删 / 改 / 查 / 按姓名排序 / 存文件 / 读文件**，菜单驱动
新学到	`struct` 定义与初始化、结构体数组、`strcpy` 赋值、**冒泡或选择排序**、**删除元素后的数组搬移**
验收标准	加 10 个人 → 删中间一个 → 排序 → 保存 → 重开 → 数据一致且有序
常见坑	① 结构体里的 `char name[20]` 不能直接赋值，要用 `strcpy` ② 删除时忘了把后面的元素往前挪，留下空洞 ③ 排序时交换的是整个结构体，不是只换姓名

做完这一级，基本就「会写程序」了。它是 C 语言课程设计的经典题，覆盖了数组、字符串、结构体、指针（排序时）、文件五个核心点。**这一级不要我给任何代码**——前面三级攒的手感，就是为了这一级能自己从零开始。

5 把通讯录拆成多文件

不增加功能 · 新维度：**工程结构** · 这是分水岭

要做的事	拆成 `main.c`（菜单和流程）+ `contact.c`（所有功能实现）+ `contact.h`（结构体定义和函数声明）+ `Makefile`
新学到	**头文件**为什么存在、**声明与定义的区别**、**编译单元**、`#include "..."` 和 `<...>` 的区别、**Makefile** 的最小写法
验收标准	只改 `contact.c` 里的一个函数，`make` 只重新编译那一个文件
常见坑	① 头文件被重复 include 导致重定义（需要 `#ifndef` 守卫）② 在 `.h` 里定义变量而不是声明 ③ 函数忘了在头文件里声明，编译时报 implicit declaration

为什么这是分水岭：单文件程序叫「脚本」，多文件 + Makefile 才叫「工程」。**从这一级开始，你写的代码才能被别人接手**——这也是为什么所有真实项目都是多文件的。

6 换语言重写（寒假）

Python 或 C++ · 新维度：**语言抽象**

做什么	用 Python 把通讯录重写一遍，**对比两种语言各花了多少行、哪些地方 C 让你痛苦**
会体会到	C 里 250 行的东西，Python 大概 80 行——因为字符串、列表、字典都是内置的。你会第一次理解**「语言是工具，不是信仰」**
验收标准	能说出三个「C 麻烦但 Python 方便」的具体例子，和两个「C 清楚但 Python 模糊」的例子

为什么要先吃 C 的苦再换：C 逼你理解内存、指针、类型——这些是所有语言的底层。先学 Python 的人往往写三年也不知道变量存在哪。**但也不能一直待在 C 里**：它没有字符串类型、没有容器，长期只用它会限制视野。

配菜：洛谷每天 3 题

项目线是主线，刷题是配菜。但配菜不能断——它维持的是「手不生」。

- **为什么是洛谷不是力扣**：力扣的 C 支持很差（没有字符串类型、没有动态数组、没有哈希表），第一题 Two Sum 就得手写哈希表，你会以为自己不会编程，其实是工具不对。
- **洛谷的好处**：官方题单是阶梯式的（【入门1】→【入门2】→…→【普及/提高-】），不用自己找题；C 语言完全够用；每题都有题解；**格式要求严格，会逼你真正掌握 scanf 的返回值和多组输入**。
- **起点**：注册后直接做 `P1001 A+B Problem`。这是所有刷题者的第一题，五行代码。**它的意义不在题目本身，在于把状态从「还没开始」改成「已经开始了」**。
- **题单路径**：【入门1】顺序结构（15 题）→【入门2】分支结构（15 题）→【入门3】循环结构 →【入门4】数组 →【入门5】字符串 →【算法1-1】模拟 →【算法1-2】排序。
- **节奏**：每天 3 题，10 月起步阶段只做前两个题单。**先写项目，再做 3 题**——项目有「今天做出来了」的即时反馈，能喂养动力；刷题容易变成机械重复。

排进日程（任务量，不是时长）

编程这学期没有死线（C 语言是上学期课，已结课），所以它的定位是**不断线**，不是主攻。

时间	项目线	刷题	定位
10 月	第 1 级（计算器）	洛谷每天 3 题，【入门1】【入门2】	起步 / 破冰
11 月	第 2 级 → 第 3 级	每天 3 题，【入门3】【入门4】	不断线
12 月	第 4 级（通讯录）	每天 1–2 题，维持手感	维持（期末优先）
寒假	第 5 级 → 第 6 级	【算法1-x】系列	**主攻**

关于「C 语言忘得差不多了」：你忘的是**语法细节**（分号、`%d` 和 `%f`、`fopen` 的参数顺序），这些查一下就回来，职业程序员也天天查文档。**没忘的是编程直觉**——变量、循环、条件、函数是什么，这些东西一旦理解就不会消失。
而且**「忘了」比「以为自己会」好教得多**。白纸状态学东西反而快，因为没有错误习惯要纠正。